

Course Outline: Mastering Control Flow in TypeScript

Title: Mastering Control Flow in TypeScript **Prerequisite:** Fundamentos de Javascript y Typescript + Tipos de datos y operadores básicos de TS (completed earlier on Day 7) **Target Audience:** Beginner developers who have just learned TypeScript fundamentals, variables, data types, and operators **Total Lessons:** 18 **Estimated Total Length:** ~9,500 words **Format:** Practical (50% hands-on)

Course Description

Now that you understand TypeScript's syntax, variables, and operators, it's time to make your programs intelligent and dynamic. Control flow is what transforms static code into programs that can make decisions, repeat actions, and respond to different conditions—the foundation of all programming logic.

This course focuses exclusively on mastering conditional statements and loops in TypeScript. You'll learn when to use if/else versus switch statements, how to choose between for and while loops, and how to control loop execution with break and continue. Most importantly, you'll learn to avoid common control flow mistakes that lead to buggy, unreadable code.

By the end of this course, you'll write programs that make complex decisions and handle repetitive tasks efficiently—essential skills for every programmer.

Course Philosophy

- This course emphasizes:
- **Converting pseudocode to TypeScript syntax:** Translating logical thinking into working code
 - **Learning through investigation:** Building the habit of researching when needed, not memorizing
 - **Writing code, not copying it:** Developing muscle memory and true understanding
 - **Identifying edge cases:** Recognizing boundary conditions that break logic
 - **Attention to detail:** Reading error messages carefully and writing one instruction per line
 - **Path of least resistance:** Starting with simple conditionals before nested complexity

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome to Control Flow	1	~450
01 - Conditional Logic Fundamentals	3	~1,700
02 - Loop Fundamentals	4	~2,300
03 - Controlling Loop Execution	3	~1,700
04 - Control Flow Anti-Patterns	2	~1,000
05 - Practice and Assessment	4	~2,400
06 - Control Flow in Real Programming	1	~450
Total	18	~9,500

Section 00: Welcome to Control Flow

00.0 Welcome to Programming Control Flow 📖 (~450 words)

Learning Objectives:

- Understand what control flow means in programming
- Recognize the two main types of control flow: conditionals and loops
- See how control flow makes programs dynamic and intelligent
- Connect control flow to everyday decision-making processes

Content Outline:

- What is control flow? The path your program takes through code
- Why static code isn't enough: programs need to make decisions
- The two pillars: conditional execution (if/switch) and repeated execution (loops)
- Real-world analogies: traffic lights (conditionals) and assembly lines (loops)
- How control flow connects to the operators you just learned
- Preview: from simple if statements to complex nested logic

Transitions: This lesson sets the foundation for all control flow concepts. You already know how to write variables and use operators—now you'll learn how to make your code respond intelligently to different situations.

Key Principles:

- Control flow determines the order in which your code executes
- Not all code runs sequentially—sometimes we skip, repeat, or choose
- Good control flow makes programs flexible and powerful
- Understanding control flow is essential for algorithmic thinking

Examples:

- A calculator app that behaves differently based on which button you press
 - A game that loops until the player wins or loses
 - A form that validates input before submitting
 - A shopping cart that applies discounts based on conditions
-

Section 01: Conditional Logic Fundamentals

01.0 Making Decisions with if Statements 📖 (~500 words)

Learning Objectives:

- Understand how if statements control code execution
- Write basic if statements with boolean conditions
- Recognize when code should execute conditionally vs. always
- Apply comparison and logical operators in conditions

Content Outline:

- The if statement syntax in TypeScript
- How boolean expressions determine execution
- Writing conditions using comparison operators (===, !==, <, >, <=, >=)
- Combining conditions with logical operators (&&, ||, !)
- Code blocks: what executes when the condition is true
- The importance of strict equality (===) over loose equality (==)
- When to use if vs. just writing sequential code

Transitions: You've learned about boolean values and comparison operators. Now you'll use them to make your programs smart enough to choose different paths based on conditions.

Key Principles:

- An if statement only executes its code block when the condition is true
- Conditions must evaluate to a boolean (true or false)
- Use strict equality (===) to avoid type coercion bugs
- Keep conditions readable—complex logic should be broken into variables

Examples:

- Checking if a user is old enough to access content
 - Displaying a warning message when a value is too high
 - Enabling a button only when required fields are filled
 - Applying a discount when a purchase meets minimum requirements
-

01.1 Building Complex Conditions with else if and else 📖 (~600 words)

Learning Objectives:

- Extend if statements with else if and else clauses
- Handle multiple distinct conditions in a single decision structure
- Understand the execution order of conditional chains
- Choose between multiple if statements vs. if/else if chains

Content Outline:

- Adding else clauses for "otherwise" logic
- Using else if for multiple exclusive conditions
- Execution order: top-to-bottom, first-match-wins
- When to use separate if statements vs. if/else if chains
- Common patterns: range checking, category assignment, validation
- Writing clear, semantic condition names
- Avoiding overly complex conditions

Transitions: Simple if statements handle one scenario, but real programs often need to choose between multiple possibilities. You'll learn how to structure multi-path decisions clearly.

Key Principles:

- else if lets you check multiple conditions sequentially
- Only one branch executes—the first one that matches
- else catches everything that didn't match earlier conditions
- Order matters: put most specific conditions first

Examples:

- Grading system: A, B, C, D, F based on score ranges
- Traffic light logic: different actions for red, yellow, green
- User role permissions: admin, editor, viewer
- Temperature warnings: too cold, perfect, too hot

Hands-On Exercise: Create a simple grade calculator that takes a numeric score and outputs the letter grade:

- 90+: "A"
- 80-89: "B"
- 70-79: "C"
- 60-69: "D"
- Below 60: "F"

Practice edge cases: What happens at exactly 90? At 59.9?

Success Criteria:

- Correctly handles all score ranges
 - Uses else if for mutually exclusive conditions
 - Tests boundary values (89, 90, 60, 59)
 - Code is readable with clear condition logic
-

01.2 Switch Statements for Multiple Cases (~600 words)

Learning Objectives:

- Understand when switch statements are clearer than if/else if chains
- Write switch statements with multiple cases
- Use break to control case execution
- Recognize switch statement limitations and alternatives

Content Outline:

- Switch statement syntax in TypeScript
- How case matching works (strict equality)
- The critical role of break statements
- Fall-through behavior when break is omitted
- Using default for unmatched cases
- When switch is clearer than if/else if
- When if/else if is better than switch
- Common use cases: menu selections, state machines, action handlers

Transitions: When you're comparing a single value against many possible matches, switch statements offer cleaner syntax than long if/else if chains.

Key Principles:

- Switch statements test one expression against multiple values
- Each case needs a break to prevent fall-through
- Default acts like else—handles unmatched cases
- Switch only works with primitive values (no ranges, no complex conditions)
- Choose switch for value-matching, if/else for range/complex logic

Examples:

- Command menu: different actions for "save", "load", "quit"
- Day of week logic: different messages for each day
- HTTP status code handling: 200, 404, 500, etc.
- Game action handler: "attack", "defend", "heal", "run"

Hands-On Exercise: Build a simple calculator that takes an operator string and two numbers:

- "+" → addition
- "-" → subtraction
- "*" → multiplication
- "/" → division
- default → "Unknown operator"

Use a switch statement to determine which operation to perform.

Success Criteria:

- Implements all four basic operations
- Uses switch statement with proper breaks

- Includes default case for invalid operators
 - Tests multiple operators to verify correctness
-

Section 02: Loop Fundamentals

02.0 Introduction to Loops and Iteration (~500 words)

Learning Objectives:

- Understand why programs need repetition
- Recognize the three components of a loop: initialization, condition, update
- Differentiate between counting loops and condition-based loops
- Identify situations where loops are necessary

Content Outline:

- What is iteration? Repeating actions until a goal is met
- Why loops matter: avoiding code duplication (DRY principle)
- The anatomy of a loop: start point, stopping condition, step
- Two loop families: for (counting) vs. while (conditional)
- Loop control flow: checking condition, executing body, updating state
- Preventing infinite loops: ensuring conditions eventually become false
- Thinking iteratively: how many times? until what condition?

Transitions: You've learned to make single decisions with conditionals. Now you'll learn to repeat actions multiple times—one of programming's most powerful tools.

Key Principles:

- Loops automate repetition instead of copy-pasting code
- Every loop needs a way to eventually stop (termination condition)
- Loop state changes each iteration to work toward the end condition
- Choose loop type based on whether you know iteration count upfront

Examples:

- Processing every item in a shopping cart
 - Counting from 1 to 100
 - Asking for user input until valid data is entered
 - Generating a multiplication table
 - Drawing repeated graphics elements
-

02.1 For Loops: Counting and Controlled Repetition (~600 words)

Learning Objectives:

- Write for loops with proper initialization, condition, and increment
- Use for loops when iteration count is known
- Access loop counter variables within the loop body
- Understand common for loop patterns and variations

Content Outline:

- For loop syntax: initialization; condition; update
- The three parts explained: let $i = 0$; $i < 10$; $i++$
- How for loops execute: check condition → run body → update → repeat
- Common patterns: counting up, counting down, skipping values

- Using the loop counter variable inside the loop
- Loop boundaries: inclusive vs. exclusive (< vs. <=)
- Off-by-one errors and how to avoid them
- When to use for loops vs. other loop types

Transitions: For loops are your tool for controlled repetition when you know exactly how many times to iterate. Master the three-part syntax and you'll handle most counting scenarios.

Key Principles:

- Use for loops when you know the iteration count upfront
- The loop counter (i, j, k by convention) tracks current iteration
- Condition is checked before each iteration—loop stops when false
- Increment (i++) moves you toward the end condition
- Keep loop variables scoped with let (not var)

Examples:

- Print numbers 1 through 10
- Calculate the sum of numbers from 1 to N
- Generate a times table for any number
- Count down from 10 to 1 (like a launch sequence)

Hands-On Exercise: Write three for loops:

1. Print even numbers from 0 to 20
2. Count backward from 100 to 50 (by 5s)
3. Calculate the factorial of a number (e.g., $5! = 5 \times 4 \times 3 \times 2 \times 1$)

Success Criteria:

- All three loops use correct for syntax
- Even numbers loop increments by 2
- Countdown loop decrements correctly
- Factorial calculation multiplies properly each iteration

02.2 While Loops: Condition-Based Iteration (~600 words)

Learning Objectives:

- Write while loops that execute based on conditions
- Choose between for loops and while loops appropriately
- Prevent infinite loops by ensuring conditions change
- Update loop state within the loop body

Content Outline:

- While loop syntax: `while (condition) { body }`
- How while loops differ from for loops: emphasis on condition, not counting
- When to use while: unknown iteration count, waiting for events
- Ensuring termination: condition must eventually become false
- Common patterns: input validation, searching, game loops
- The danger of infinite loops: condition never changes
- Comparing while vs. for: when each is clearer
- Initialization and update are manual (not built into syntax)

Transitions: For loops excel at counting, but what if you don't know how many iterations you need? While loops let you repeat until a condition changes—perfect for validation, searching, and unpredictable scenarios.

Key Principles:

- While loops run as long as the condition is true
- You must manually update variables to change the condition
- Use while when you don't know iteration count upfront
- Every while loop needs a way to eventually stop
- Initialize relevant variables before the loop

Examples:

- Keep asking for input until user enters valid data
- Roll dice until you get a 6
- Find the first number divisible by 7 after 100
- Game loop: keep playing until player chooses to quit

Hands-On Exercise: Create a number guessing game simulation:

- Secret number is 42
- Start with guess = 0
- While guess is not 42, generate random guesses
- Count how many attempts it took
- Print the count after the loop

Success Criteria:

- While loop checks if guess !== 42
 - Guess is updated each iteration (simulate new random number)
 - Counter increments each loop iteration
 - Loop terminates when guess equals 42
 - Final attempt count is printed
-

02.3 Advanced Loop Variants: for...in, for...of, and do-while 📖 (~600 words)

Learning Objectives:

- Understand for...in and for...of syntax for iterating collections
- Recognize when to use do-while instead of while
- Differentiate between the four loop types
- Choose the appropriate loop for each scenario

Content Outline:

- for...in loops: iterating object keys/indices
 - Syntax: for (let key in object)
 - Use case: when you need property names
 - Caution: order not guaranteed
- for...of loops: iterating collection values
 - Syntax: for (let value of array)
 - Use case: when you only need values, not indices
 - Cleaner than traditional for loops for arrays
- do-while loops: execute at least once
 - Syntax: do { body } while (condition)
 - Condition checked after body execution
 - Use case: guaranteed first execution (menus, prompts)
- Comparison chart: when to use each loop type
- Modern preference: for...of for arrays, while for conditions

Transitions: Beyond traditional for and while loops, TypeScript offers specialized loop syntax for common patterns. These variants make code cleaner and more expressive for specific scenarios.

Key Principles:

- for...in iterates keys/indices (objects and arrays)
- for...of iterates values (arrays and other iterables)
- do-while guarantees at least one execution
- Choose the loop that matches your intent most clearly
- Modern code prefers for...of over traditional for for arrays

Examples:

- for...in: listing all properties of an object
 - for...of: printing each item in an array
 - do-while: displaying a menu and waiting for valid input
 - Comparison: four ways to iterate an array
-

Section 03: Controlling Loop Execution

03.0 Flow Interruption: break and continue 📖 (~500 words)

Learning Objectives:

- Understand how break immediately exits a loop
- Use continue to skip the current iteration
- Recognize appropriate use cases for each keyword
- Avoid misusing break/continue in place of better logic

Content Outline:

- What break does: terminates the loop immediately
- What continue does: skips to next iteration
- When to use break: early exit when goal is achieved
- When to use continue: skipping invalid/unwanted iterations
- How break/continue affect loop control flow
- Difference between break and return
- Common patterns: search-and-stop, filtering during iteration
- Warning: overusing break/continue can reduce readability

Transitions: Sometimes you need to stop a loop early or skip certain iterations. Break and continue give you fine-grained control over loop execution—but use them wisely.

Key Principles:

- break exits the entire loop immediately
- continue skips to the next iteration (condition check)
- Use break when you've found what you're looking for
- Use continue to skip processing for specific values
- Prefer clear loop conditions over excessive break/continue
- break only exits the innermost loop (in nested loops)

Examples:

- Search array for a value, break when found
- Skip negative numbers when processing data
- Exit loop when user types "quit"
- Continue past invalid entries in a list

03.1 Mastering break and continue in Practice (~600 words)

Learning Objectives:

- Apply break to stop loops early when conditions are met
- Use continue to filter iterations selectively
- Combine break/continue with conditionals effectively
- Write cleaner code by choosing the right control flow tool

Content Outline:

- Practical break scenarios: finding matches, reaching limits
- Practical continue scenarios: skipping errors, filtering values
- Combining with if statements: conditional interruption
- The pattern: search algorithms use break heavily
- The pattern: data processing uses continue for filtering
- When break/continue make code clearer
- When they make code more confusing (prefer better conditions)
- Testing: ensuring loops behave correctly with interruptions

Transitions: Theory is essential, but control flow mastery comes from practice. You'll implement real scenarios where break and continue solve problems elegantly.

Key Principles:

- break for "I found it, stop searching"
- continue for "skip this one, keep going"
- Always consider if clearer condition logic would eliminate the need
- Comment why you're breaking/continuing if it's not obvious

Examples:

- Find first prime number after 100 (use break)
- Sum only positive numbers in an array (use continue)
- Process items until reaching a "STOP" marker
- Skip weekends when calculating business days

Hands-On Exercise: Write two programs:

1. Find the first number divisible by both 3 and 7 between 1 and 100
 - Use a for loop
 - Use break when you find it
2. Sum all numbers from 1 to 50, but skip numbers divisible by 3
 - Use a for loop
 - Use continue to skip multiples of 3

Success Criteria:

- First program stops immediately after finding the number
- Second program correctly excludes multiples of 3
- Both use appropriate break/continue keywords
- Code includes console.log to verify results

03.2 Nested Loops: Loops Within Loops (~600 words)

Learning Objectives:

- Write nested loops with proper variable naming
- Understand how nested loops multiply iterations
- Use nested loops for multi-dimensional data
- Control which loop break affects in nested structures

Content Outline:

- What are nested loops? Loops inside loops
- How nested loops execute: outer loop controls inner loop
- Iteration multiplication: outer × inner total executions
- Variable naming: i, j, k convention for nested counters
- Common patterns: multiplication tables, matrix operations, combinations
- Breaking out of nested loops: which loop exits?
- Performance considerations: nested loops can be slow
- When to use nested loops vs. alternative approaches

Transitions: Single loops handle one-dimensional repetition, but real problems often involve two-dimensional thinking: rows and columns, x and y coordinates, combinations of items. Nested loops are your tool for these scenarios.

Key Principles:

- Outer loop controls how many times the inner loop fully executes
- Use distinct counter variables (i for outer, j for inner, k for third level)
- Total iterations = outer iterations × inner iterations
- break only exits the innermost loop unless you use labels (advanced)
- Keep nesting shallow—more than 2-3 levels is usually bad design

Examples:

- Generating a multiplication table (1-10 × 1-10)
- Printing a grid or matrix pattern
- Finding all pairs of numbers that sum to a target
- Nested menus: categories → items within category

Hands-On Exercise: Create three nested loop programs:

1. Print a 5×5 grid of asterisks:

```
*****
*****
*****
*****
*****
```

2. Generate a right triangle pattern:

```
*
**
***
****
*****
```

3. Find all pairs (i, j) where i + j = 10 and both are positive integers

Success Criteria:

- Grid uses nested loops (outer for rows, inner for columns)
- Triangle uses nested loop with increasing inner iterations
- Pairs program correctly identifies all valid combinations

- Code is properly indented to show nesting
-

Section 04: Control Flow Anti-Patterns

04.0 Anti-Pattern: Arrow Code and Excessive Nesting 📖 (~500 words)

Learning Objectives:

- Recognize "arrow code" and why it's problematic
- Understand how excessive nesting reduces readability
- Learn techniques to flatten nested conditionals
- Apply early returns and guard clauses to simplify logic

Content Outline:

- What is arrow code? Deep nesting that creates visual arrows (>>)
- Why excessive nesting is bad: hard to read, hard to test, error-prone
- The cognitive load problem: humans can only track 3-4 levels
- How arrow code happens: adding if inside if inside if
- Refactoring technique: early returns (guard clauses)
- Refactoring technique: extract conditions to variables
- Refactoring technique: invert conditions to reduce nesting
- The rule: keep nesting under 3 levels whenever possible

Transitions: One of the most common mistakes beginners make is nesting conditionals too deeply. You'll learn to recognize this anti-pattern and refactor it into cleaner, more maintainable code.

Key Principles:

- Deep nesting makes code hard to understand and maintain
- Each nesting level adds cognitive complexity
- Early returns let you handle edge cases first, then main logic
- Clear variable names for conditions reduce need for nesting
- Flat code is easier to read than deeply nested code

Examples:

- Bad: `if (x) { if (y) { if (z) { ... } } }`
- Good: `if (!x) return; if (!y) return; if (!z) return; ...`
- Bad: nested conditionals for validation
- Good: guard clauses at function start
- Refactoring demonstration: 4-level nesting → 1-level with guards

Anti-Pattern Indicators:

- Visual arrow shape in your code (>>>>)
- Struggling to track which "}" closes which condition
- Needing to scroll horizontally to read logic
- Four or more indentation levels

Correct Approach:

- Handle special cases first with early returns
 - Use descriptive variable names for complex conditions
 - Break complex conditionals into separate if statements
 - Keep main logic at the top level when possible
-

04.1 Anti-Pattern: Infinite Loops and Common Mistakes 📖 (~500 words)

Learning Objectives:

- Understand how infinite loops occur
- Recognize conditions that never change
- Identify off-by-one errors in loop boundaries
- Debug common loop mistakes before they crash your program

Content Outline:

- What is an infinite loop? Condition never becomes false
- Common cause #1: forgot to update counter/variable
- Common cause #2: wrong comparison operator in condition
- Common cause #3: updating wrong variable
- Off-by-one errors: < vs. <=, starting at 0 vs. 1
- The "loop runs one too many/few times" problem
- Debugging infinite loops: how to spot and fix them
- Prevention: always verify loop will terminate before running

Transitions: Even experienced programmers sometimes create infinite loops. Learning to recognize and prevent them saves hours of debugging frustration.

Key Principles:

- Every loop must have a way to terminate
- The loop condition must be affected by the loop body
- Check your loop boundaries carefully (off-by-one errors are common)
- When in doubt, add console.log to track iterations
- While loops are more prone to infinite loops than for loops

Examples:

- Infinite: `while (x > 0) { console.log(x) } // never changes x`
- Fixed: `while (x > 0) { console.log(x); x-- }`
- Off-by-one: `for (let i = 0; i <= 10; i++) // runs 11 times, not 10`
- Wrong operator: `while (x = 5) // assignment, always true!`

Anti-Pattern Indicators:

- Program freezes or browser tab becomes unresponsive
- Loop counter never updates inside loop body
- Using `=` instead of `===` in condition
- Decrementing when you should increment (or vice versa)

Correct Approach:

- In while loops, ensure the condition variable changes in the body
- Use for loops when iteration count is known (less error-prone)
- Test loop boundaries with small numbers first
- Add temporary console.log to verify iterations before running on large data

Section 05: Practice and Assessment

05.0 Conditional Logic Practice Challenge 📖 (~600 words)

Learning Objectives:

- Apply conditional logic to solve real-world scenarios
- Choose between if/else if and switch statements appropriately
- Handle edge cases and boundary conditions
- Write clean, readable conditional code

Content Outline:

- Challenge scenarios requiring conditional logic
- Combining comparison and logical operators
- Handling input validation with conditionals
- Testing all branches of your conditional logic
- Writing semantic condition names for clarity

Transitions: You've learned the theory and seen examples—now it's time to solve problems independently. These challenges will test your understanding of conditional logic in practical scenarios.

Key Principles:

- Read requirements carefully before coding
- Test edge cases and boundary values
- Keep conditions simple and readable
- One condition per line when possible for clarity

Hands-On Exercise: Solve these three conditional challenges:

Challenge 1: Shipping Calculator Calculate shipping cost based on weight and destination:

- Domestic: \$5 base + \$0.50/kg
- International: \$15 base + \$2/kg
- Free shipping for orders over \$100 (before shipping)
- Express option adds \$10

Challenge 2: Password Validator Check if a password is valid:

- At least 8 characters
- Contains at least one number
- Contains at least one uppercase letter
- Not in list of common passwords ["password", "123456", "qwerty"]

Challenge 3: Ticket Pricing Calculate movie ticket price:

- Children (under 12): \$8
- Seniors (65+): \$10
- Adults: \$15
- Matinee (before 5pm): 20% off
- Weekend: \$2 surcharge

Success Criteria:

- All edge cases handled correctly
- Code uses appropriate conditional structures
- Conditions are readable and well-named
- All branches tested with sample inputs

05.1 Loop Mastery Practice Challenge (~600 words)

Learning Objectives:

- Apply loop concepts to solve algorithmic problems

- Choose appropriate loop types for different scenarios
- Use break and continue effectively
- Write efficient, readable loop code

Content Outline:

- Challenge scenarios requiring iteration
- Selecting for vs. while based on problem requirements
- Accumulating values across iterations
- Early exit strategies with break
- Selective processing with continue

Transitions: Loops are fundamental to algorithms and data processing. These challenges will develop your loop-thinking skills and your ability to choose the right loop for the job.

Key Principles:

- Identify what needs to repeat before choosing loop type
- Initialize accumulators before the loop
- Verify termination conditions will eventually be met
- Test with small inputs before scaling up

Hands-On Exercise: Solve these three loop challenges:

Challenge 1: Prime Checker Write a function that checks if a number is prime:

- Test divisibility from 2 to $\sqrt{n}$
- Use break when a divisor is found
- Return true if no divisors found

Challenge 2: Fibonacci Sequence Generate the first N Fibonacci numbers:

- Start with 0, 1
- Each next number is sum of previous two
- Store in array or print each
- Example: $N=7 \rightarrow 0, 1, 1, 2, 3, 5, 8$

Challenge 3: Digital Root Find the digital root of a number (sum digits repeatedly until single digit):

- Example: $38 \rightarrow 3+8=11 \rightarrow 1+1=2$
- Use nested loops or loop with calculation
- Test with: 16, 942, 12345

Success Criteria:

- Correct loop types chosen for each problem
- Edge cases handled ($N=0$, $N=1$, prime numbers)
- Code is efficient (no unnecessary iterations)
- Clear variable names explain intent

05.2 Combined Control Flow Exercises (~600 words)

Learning Objectives:

- Combine conditionals and loops to solve complex problems
- Use nested structures appropriately
- Apply break/continue in context
- Write complete programs with proper control flow

Content Outline:

- Problems requiring both conditionals and loops
- Managing state across iterations
- Conditional loop termination
- Nested loop scenarios
- Building complete solutions with proper structure

Transitions: Real programs rarely use just conditionals or just loops—they combine both. These final exercises challenge you to integrate everything you've learned.

Key Principles:

- Break problems into smaller steps
- Use conditionals inside loops when each iteration needs decisions
- Use loops inside conditionals when certain cases need repetition
- Keep nesting reasonable (2-3 levels maximum)

Hands-On Exercise: Solve these integrated challenges:

Challenge 1: Number Pyramid Print a number pyramid:

```

  1
 121
12321
1234321
123454321

```

Requirements:

- Outer loop for rows
- Inner loops for spaces and numbers
- Conditional logic for counting up vs down in each row

Challenge 2: Grade Statistics Process a list of test scores (given as array):

- Count how many A, B, C, D, F grades
- Calculate average score
- Find highest and lowest scores
- Stop processing if score is negative (invalid data marker)

Challenge 3: Simple ATM Simulator Simulate ATM operations:

- Start with balance of \$1000
- Loop menu: Check Balance, Deposit, Withdraw, Exit
- Validate: no negative deposits, can't withdraw more than balance
- Continue until user selects Exit

Success Criteria:

- Programs run without errors
- All requirements implemented correctly
- Control flow is clear and logical
- Edge cases handled appropriately

05.3 Control Flow Mastery Assessment 🧠 (~600 words)

Learning Objectives:

- Demonstrate understanding of conditional logic concepts
- Show mastery of loop types and their applications

- Analyze code to identify control flow patterns
- Evaluate and debug control flow problems

Assessment Format: Scenario-based questions

Question 1: Conditional Logic Analysis Given this code:

```
let score = 85;
let grade: string;

if (score >= 90) {
  grade = "A";
} else if (score >= 80) {
  grade = "B";
} else if (score >= 70) {
  grade = "C";
} else {
  grade = "F";
}
```

Scenario questions:

- What grade is assigned when score is 85?
- What happens if score is exactly 90?
- What's wrong with this code? (Hint: missing grade)
- How would you refactor this using a switch statement?
- What grade is assigned if score is 65?

Question 2: Loop Type Selection For each scenario, identify the best loop type (for, while, do-while, for...of) and explain why:

a) Print all even numbers from 0 to 100 b) Keep asking for password until correct c) Process each item in a shopping cart array d) Display a menu at least once, then repeat until user quits e) Find the first number divisible by 17 between 1000 and 2000

Question 3: Debugging Infinite Loops Identify why these loops are infinite and how to fix them:

a)

```
let count = 10;
while (count > 0) {
  console.log(count);
}
```

b)

```
for (let i = 0; i < 10; i--) {
  console.log(i);
}
```

c)

```
let found = false;
while (!found) {
  console.log("Searching...");
  if (Math.random() > 0.5) {
    console.log("Found!");
  }
}
```


Question 4: Nested Loop Iteration Count How many times does the inner loop body execute in total?

```
for (let i = 0; i < 4; i++) {  
  for (let j = 0; j < 3; j++) {  
    console.log(i, j);  
  }  
}
```

Options: A) 4 B) 7 C) 12 D) 16

Question 5: Break vs Continue Predict the output of this code:

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    continue;  
  }  
  if (i === 5) {  
    break;  
  }  
  console.log(i);  
}
```

Question 6: Anti-Pattern Recognition Which code sample demonstrates arrow code anti-pattern? What's wrong with it and how would you fix it?

Question 7: Practical Application Write pseudocode (not actual code) for this scenario: "Create a program that asks users to guess a number between 1-100. After each guess, tell them if they're too high or too low. Continue until they guess correctly. Count the number of attempts."

Evaluation Criteria:

- **90-100%:** Mastery - Correctly answers all conceptual and application questions
- **80-89%:** Proficient - Minor mistakes but demonstrates strong understanding
- **70-79%:** Developing - Understands basics but struggles with complex scenarios
- **Below 70%:** Needs review - Revisit key concepts before proceeding

Assessment Purpose: This assessment verifies your readiness to use control flow in real programming scenarios. It's not about memorizing syntax—it's about understanding when and why to use each control structure.

Section 06: Control Flow in Real Programming

06.0 Building with Control Flow (~450 words)

Content Outline:

- Recap: You've mastered the foundations of program logic
- What you can now build:
 - Programs that make intelligent decisions
 - Code that processes data efficiently
 - Algorithms that solve real problems
 - Interactive applications with user input
- How control flow connects to your programming journey:
 - Next: functions to organize and reuse control flow
 - Soon: arrays and objects to manage complex data with loops
 - Eventually: combining all these concepts in full applications
- Real-world impact: every program you use relies on control flow

- Form validation before submission
- Game loops that run until win/lose condition
- Search algorithms that stop when finding results
- Data processing that filters and transforms information
- Best practices to carry forward:
 - Keep conditions readable and simple
 - Choose the right loop for the job
 - Avoid excessive nesting
 - Test edge cases and boundaries
 - Write code that clearly expresses intent
- Resources for continued learning:
 - Practice algorithmic problems on coding challenge sites
 - Read others' code to see different control flow approaches
 - Experiment with nested structures gradually
 - Always test your loops with small inputs first
- Encouragement:
 - Control flow feels complex at first—that's normal
 - Every programmer has written infinite loops during learning
 - The more you practice, the more natural it becomes
 - You now have the fundamental building blocks of programming logic
- Next steps:
 - Apply these concepts in upcoming function lessons
 - Combine control flow with data structures
 - Build increasingly complex programs
 - Develop your algorithmic thinking skills

Note: This is a conclusion lesson only—no new theory, no exercises, no assessment. It's a moment to reflect on what you've accomplished and look ahead to how these skills will serve your programming journey.

End of Course Outline